

Competitive Esports Network: A Relational Approach to Tournament Management

Phase IV

Wyatt Fischer
fisc7842@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Nastia Kossiak
koss0519@vandals.uidaho.edu
University of Idaho
Moscow, Idaho, USA

Brandon Lunney
lunn2076@vandals.uidaho.edu
University of Idaho
Moscow, Idaho USA

Abstract

The final report for the E-Sports Competitive Network is presented. A tool which is designed to showcase player, team, and tournament's dynamic relationships through a graphical interface and underlying relational database. The report includes a finalized prototype description detailing all the features included within the finished product. The finalized entity relation model with discussion on changes from the initial design. A finalized tech stack from the database to the front end. Implementation details for each of the custom web scrappers that populate the database. Discussion on each of the three core analytical queries provided by the tool. Including showcases of all web pages of the project. Lastly, a reflection on the initial timeline with a phase by phase break down on the project's development.

CCS Concepts

• Relational Databases; • Computer Games;

Keywords

Esports, Relational Databases, MySQL, Cytoscape, Network Visualization, Python

ACM Reference Format:

Wyatt Fischer, Nastia Kossiak, and Brandon Lunney. 2024. Competitive Esports Network: A Relational Approach to Tournament Management Phase IV. In . ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/nnnnnnn>

1 Introduction

E-Sports is one of the fastest growing media in recent years. The biggest e-sports events can even receive upwards of 5.5 million views. With a wide range of games, including titles such as *League of Legends*, *Counter-Strike*, and *Dota 2*, the industry has developed a vast ecosystem of professional teams and large-scale tournaments. However, unlike it's real life counterpart e-sports does not have near the amount of statistical analysis tools as regular sports. These

tools allow coaches, fans, and researchers to better understand performance trends, strategic strengths, and the relationships between teams across seasons. This was the motivation behind the *E-Sports Competitive Network*. For this project we will be implementing a relational database to facilitate statistical analysis of e-sports players and teams across several seasons and games. We will then utilizing that database to visually communicate the information in a graph format.

The requirements of this project are we facilitate multiple types of nodes, in this instance this is players, teams, and the tournaments themselves. This project will also allow for several edges types, such that teammates, played in, and played for. We also allow for multi-graph support, specifically the ability for a node to belong to several graphs and by extension edges to multiple graphs as well. There will also feature a minimum set of queries available, such as an h-index filter or for our purposes a win rate filter for the teams and players. As well as a Co-Author network which for this system will be a teammate network. Lastly, there will be a Q1 journal influence network. However, for our purposes this will be a Q1 tournament influence network, where the Q1-Q4 will be determined based on which quartile the prize pool falls into. Then we will find all teams and by extension the players that participated in that tournament.

2 Final Prototype Description

This report covers the final prototype of the Competitive Esports Network. The E-Sports Competitive Network final prototype is designed to address the lack of advanced statistical analysis tools in competitive gaming. This prototype introduces a relational database system that models graph connections between players, teams, and tournaments across multiple games. This prototypes implements three key queries which can be applied onto the project's database. First, we have a teammate network query. This query was based off the *Co-Author Network* query seen in several journal analytical tools, allowing for a visualization of player interactions and relationships. The second query this project includes is an h-index filter. Based on the analytical query with the same name this query allows for both tournament participation and tournament wins filters. Allowing for an analytical analysis of team performance. The final query is a Q1 tournament influence network, which is based on a Q1 journal influence network. Which allows for tournaments to be filtered into Q1 status and then get all teams that participated in that tournament. Allowing for a measurement of a tournament's impact. Aside from the three primary queries the final prototype features several web pages to display the contents of the queries as

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/10.1145/nnnnnnn>

well as allow for input into the database manually. We have also built several web scrappers to populate the database with data.

3 Final Entity Relation Model

The final entity relation model looks vastly different than the initial model, as seen in figure 1. Initially we planned on having each different type of data, player, team, and tournament, represent its own table. In addition with the respective relationships between the data points. This worked fine for the basic prototype which allowed for basic create, read, update, and delete operations within the database. However, as we implemented more of the graph front end this schema did not hold up. This lead to complex queries to get the appropriate information for Cytoscape and did not allow for multi-graph support. Therefore we ended up with the final model depicted in Figure 2.

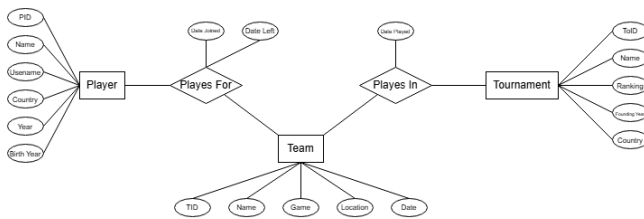


Figure 1: Initial Entity Relation Diagram

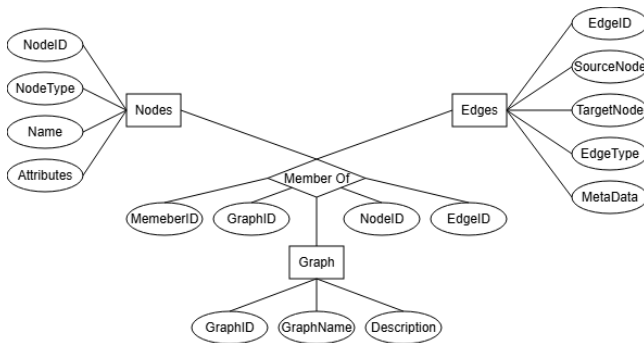


Figure 2: Finalized Entity Relation Model

The benefits of this model is rather than having three different types of entities for each team, player, and tournament we simply have a Node entity. Each Node has a NodeID which acts as the primary key. Then the team, player, or tournament attribute is stored withing the NodeType value. Since each team, player, and tournament has different amount of attributes stored with it the Attributes column is a JSON file allowing for flexible metadata on the node.

Similarly, rather than having two different tables for the played-for and played-in relationship we consolidated it to one Edge entity. Each edge has a an EdgeID acting as the primary key. As well as two node id's, SourceNodeID and TargetNodeID, which describe the nodes the edge connects as well as are foreign keys to the node table. For the same reasons as the node entity we gave the edge

entity and EdgeType value which holds the played-in or played-for relationship. Lastly, since each relationship holds a different number of descriptors edges have a MetaData value which acts identically to the Attributes column of nodes.

To facilitate the multi-graph support we've added a new entity to represent the graphs themselves. Each graph has a GraphID which is the primary key of the table. Each graph also has a name that can be displayed. In addition, each graph has a description to further add details to the graph. We've also added a new relationship between the entities, Member of, this represents the relationship between a graph and a node or edge. Each membership has it's own MemberID which acts as the primary key. A nodeID or edgeID which determines which entity belongs to the graph. The graphID to determine which graph the entity belongs to. This schema allows for easy graph integration and multi-graph support, in addition for further expansion if we decide to add more types of nodes or edges.

4 Tech Stack

4.1 Database

Initially the database we decided on was **MySQL**, which is based around the client-server model. To simplify development and improve collaboration we've transitioned to **SQLite**, a lightweight relational database that follows a serverless architecture. This design makes it extremely easy to set up and allows team members to share the database simply by distributing the file.

4.2 Back-End

For the back end we've utilized **Python** for it's expansive libraries and data analysis tools. We leveraged Python's frameworks and libraries to handle core logic and data interaction with the SQLite database. Discussed later, we utilized Flask for webpage routing and Playwright to populate the database.

4.3 Front-End

The front end utilizes HTML and CSS for the webpage templates as well as the presentation. To present the graph presentation of the network we utilize Cytoscape which is an open-source network visualizer. Which allows us to render complex relationships in an intuitive way. Through its JavaScript integration we are able to embed interactive graphs directly into the webpage.

5 Implementation

5.1 Populating the Database

To populate the initial database we've developed a web scrapper that utilizes Playwright to load web pages and scrape the information from the html tags then inserts them into the database. The scraper checks for duplicate nodes, in that case the node's attributes are updated. In addition, the scrapper creates the respective edges as well. Below is a list of webpages where we scrape the information.

- (1) <https://prosettings.net/players/>
- (2) <https://en.wikipedia.org/wiki/>
- (3) https://liquipedia.net/valorant/S-Tier_Tournaments
- (4) https://liquipedia.net/fortnite/S-Tier_Tournaments
- (5) https://liquipedia.net/dota2/Tier_1_Tournaments
- (6) https://liquipedia.net/counterstrike/S-Tier_Tournaments

- (7) https://liquipedia.net/rainbowsix/S-Tier_Tournaments
- (8) https://liquipedia.net/pubg/S-Tier_Tournaments
- (9) https://liquipedia.net/apexlegends/S-Tier_Tournaments
- (10) https://liquipedia.net/rocketleague/S-Tier_Tournaments
- (11) https://liquipedia.net/leagueoflegends/S-Tier_Tournaments

5.1.1 Player Web Scrapper. The web scrapper designed to collect Player information gathers its information from link (1), it starts by creating the respective tables if needed, Nodes and Edges. Then, after establishing the database connection, utilizes playwright to start a Mozilla 5.0 instance. Then ignores heavy resources such as images, fonts, and other media. Then utilizing a regex statement pulls finds all locations on the document that match '[a\[href*="/players/"\]](#)'. This is then cleaned to remove the player's game information information, leaving a ready to process url. The scrapper also checks if no new players are added as well. Once there are no new players added from scrapping the scrapper moves to inserting the information. Each of the grabbed urls is then loaded individually. The scrapper then pulls the table tags from the page. Then iterates through the table rows and columns collecting the player's real name, birthday, and country or Unknown if the information is null. Lastly, it collects the team the player belongs to or assumes "Free Agent" if the do not belong to a team. Once the information has been collected the following queries are executed. The first query is to check if the information has already been inserted. If it has been inserted the information is updated with the second query. If it is not present in the database the last query is executed to insert the information into the database.

```
SELECT NodeID
FROM Nodes
WHERE Name = $username
AND NodeType = 'Player';

UPDATE Nodes
SET Attributes = $player_attributes
WHERE NodeID = $player_id;

INSERT INTO Nodes
(NodeType, Name, Attributes)
VALUES ('Player', $username, $player_attributes);
```

Since many players may belong to one team the scrapper also checks to make sure the same team is not inserted several times. It does this with the below queries. The first one checks if the team has already been inserted, in the event it has not been inserted the second query is executed.

```
SELECT NodeID
FROM Nodes
WHERE Name = $team_name
AND NodeType = 'Team'

INSERT INTO Nodes
(NodeType, Name, Attributes)
VALUES ('Team', $team_name, $team_attributes)
```

Once this has been executed the last thing the scrapper needs to do it insert relevant edge information. This is done by the below

queries, the first checks if the edge has already been inserted. If it is not present the second is executed.

```
SELECT EdgeID
FROM Edges
WHERE SourceNodeID = $player_id
AND TargetNodeID = $team_id
AND EdgeType = 'Plays_For'

INSERT INTO Edges
(SourceNodeID, TargetNodeID, EdgeType)
VALUES ($player_id, team_id, 'Plays_For');
```

This process is repeated for each of the player urls that get scrapped during the initial url collection. Once this is completed the scrapper has been completed.

5.1.2 Teams Web Scrapper. The teams web scrapper is designed to collect as well as fill in missing team information from link (2). The scrapper starts by creating the needed tables if need as well as establishing the database connection. Once that has been completed the below query is executed, which pulls teams that are missing location or founded attributes.

```
SELECT NodeID, Name, Attributes
FROM Nodes
WHERE NodeType = "Team"
AND (Attributes NOT LIKE '%"location"%')
OR Attributes NOT LIKE '%"founded"%')
```

For each of the teams with partial information it pulls what information is present then loads the respective wikipedia page. Since some pages have inconsistent naming schemes the scrapper is designed to check both `/wiki/$teamname` and `/wiki/$teamname_(esports)`. Once the page is loaded the scrapper pulls from the respective table on the page, aiming to gather the location and founder information if present. Once the wikipedia information is collected the below query is executed. If the Wikipedia page was found then the attributes are updated with the found information. If the page could not be found the attributes are updated with the `wiki_found` value set as false.

```
UPDATE Nodes
SET Attributes = $attrs
WHERE NodeID = $teamID
```

Once this is completed for all teams with missing information then the scrapper is finished.

5.1.3 Tournament Web Scrapper. Lastly the tournament web scrapper is designed to gather tournament information from links (3) through (11). Similar to the other two the scrapper starts by creating the respective tables if necessary and establishes the database connection. Playwright is then used to initialize the browser instance. Then for each of the games and it's respective url, the browser opens the game's website loading the table present on the page. For each row in the table it gathers only the tournaments that are gold highlighted, collecting the tournament's url, date, prize pool, location, and winner information. The scrapper does not collect tournaments that have not happened yet. Once all the information has been gathered the first query is executed to insert the information. If the tournament is already present the second query is

executed. Lastly the winning team is inserted into the database as well.

```
INSERT OR IGNORE INTO Nodes
(NodeType, Name, Attributes)
VALUES ('Tournament', $tournament_name, $attr_json)
```

```
UPDATE Nodes
SET Attributes = $attr_json
WHERE Name = $tournament_name
AND NodeType = 'Tournament'
```

```
INSERT OR IGNORE INTO Nodes
(NodeType, Name, Attributes)
VALUES ("Team", $winner, $game_attributes);
```

Next the scrapper moves to insert relevant edge information. The below queries are executed, the first query is used to get the appropriate node information for the given tournament from the database. Then the winning team's information is gathered with the second query. Then the scrapper adds both a *Won By* and *Played In* edge into the edges table connecting the tournament and winning team. This is accomplished by the third and fourth queries. The same process is repeated for the runner up team, but does not add the *Won By* edge. This is completed by the fifth and sixth queries. Once this is executed for every tournament for each of the games listed in the links table the web scrapper is complete.

```
SELECT NodeID
FROM Nodes
WHERE Name = $tournament_name
AND NodeType = "Tournament"
```

```
SELECT NodeID
FROM Nodes
WHERE Name = $winner
AND NodeType = "Team"
```

```
INSERT OR IGNORE INTO Edges
(SourceNodeID, TargetNodeID, EdgeType)
VALUES ($tournament_ID, $winner_ID, 'Won_By')
```

```
INSERT OR IGNORE INTO Edges
(SourceNodeID, TargetNodeID, EdgeType)
VALUES ($tournament_ID, $winner_ID, 'Played_In')
```

```
SELECT NodeID
FROM Nodes
WHERE Name = $runner_up
AND NodeType = "Team"
```

```
INSERT OR IGNORE INTO Edges
(SourceNodeID, TargetNodeID, EdgeType)
VALUES ($tournament_ID, $runner_up_ID, 'Played_In')
```

In addition to the web scraper, the project also includes several dedicated webpages that allow users to insert data directly into the database. These pages utilize forms to fill in a SQL statement that is then executed by the database manager. Figures 3 showcases the database insertion webpages.

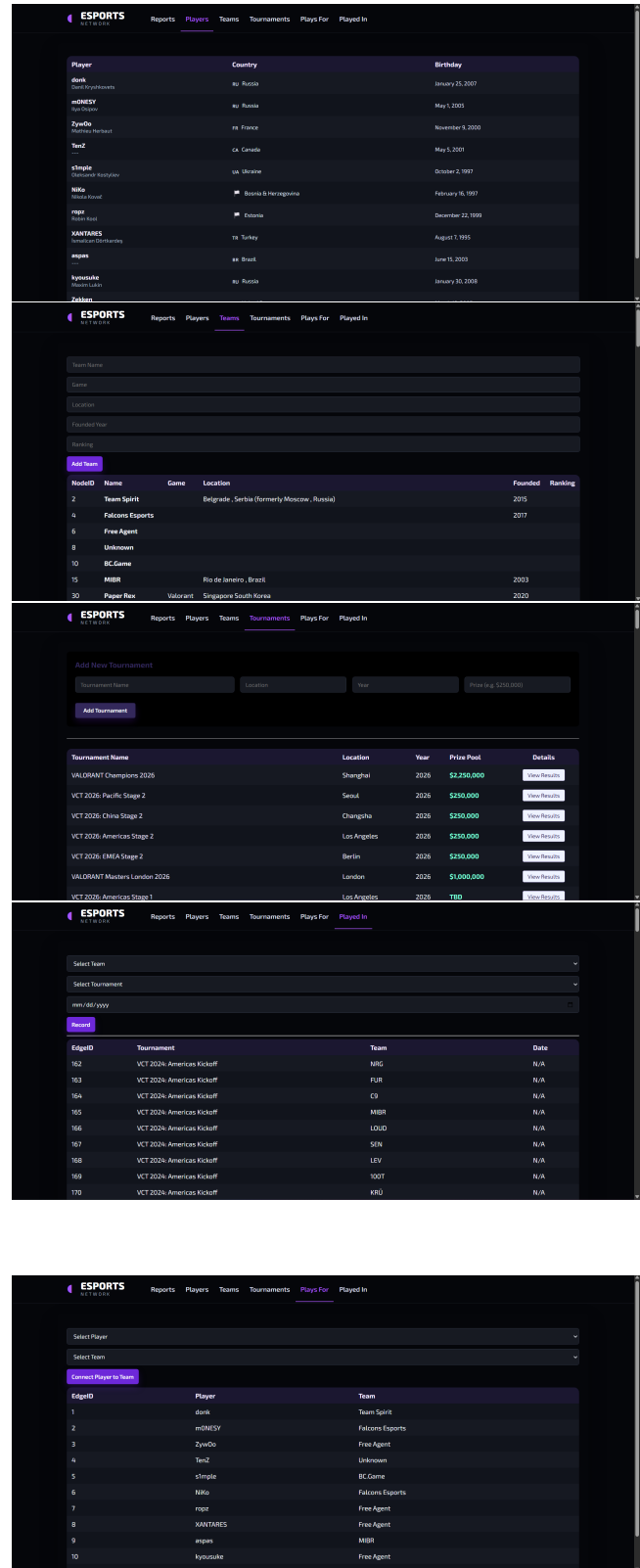


Figure 3: Database Insertion Webpages

5.2 Reports Page

As mentioned previously the project utilizes Flask to handle web-page routing. Each webpage calls from the database and then fills in the template html page with the returned information from the database. The front page get the entire database then utilizes Cytoscape's java script integration to present the information. Figure 4 showcases the front webpage.

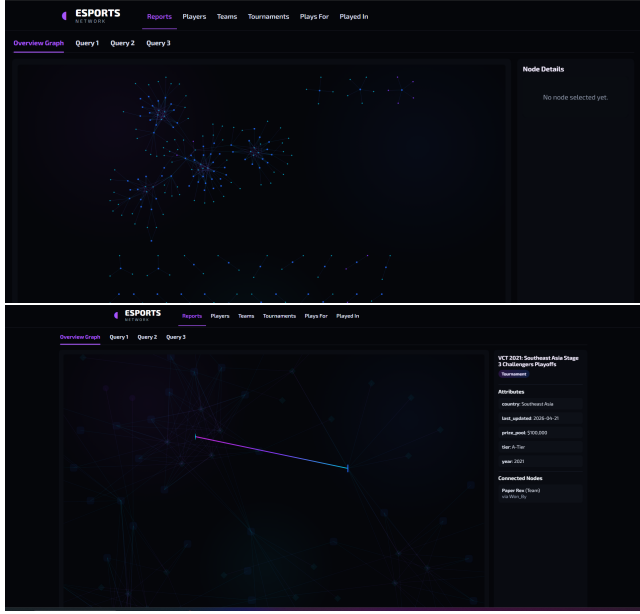


Figure 4: Reports Web Page

5.3 Query 1

For this project the first analytical query we implemented was a *Teammate Network*. This is based on a similar concept of a *Co-Author Network* used by research paper databases. This query is used to visualize the player's relationships and interactions. The below queries are executed when completing the query. Given a specific player we find all node that are connected the same matching team with the *Plays For* edge. Figure 5 showcases the query web pages themselves.

```
SELECT DISTINCT n.NodeID, n.Name, n.NodeType, n.Attributes
FROM Nodes n
JOIN GraphMemberships gm ON gm.NodeID = n.NodeID
WHERE gm.GraphID = 1
AND n.NodeType IN ("Player", "Team")
ORDER BY n.NodeType, n.Name
SELECT DISTINCT e.EdgeID, e.SourceNodeID, e.TargetNodeID,
e.EdgeType, e.Metadata
FROM Edges e
JOIN GraphMemberships gm ON gm.EdgeID = e.EdgeID
WHERE gm.GraphID = 1
AND e.EdgeType = "Plays_For"
```

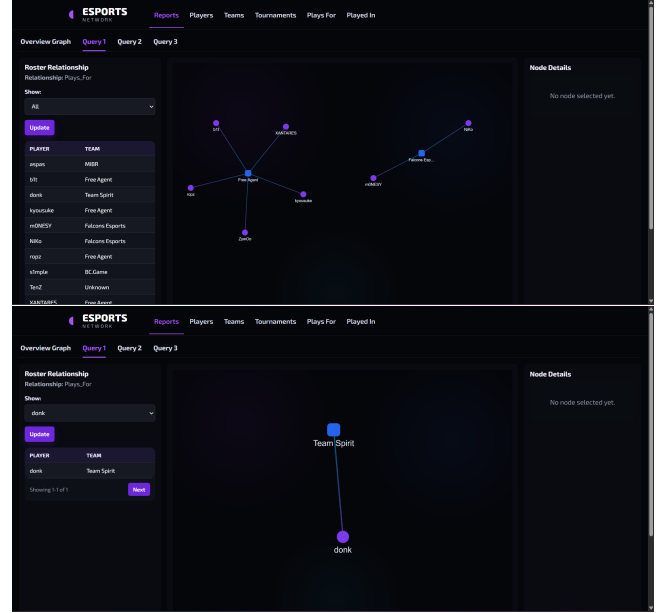


Figure 5: Query 1 Webpage with Example

5.4 Query 2

For the second analytical query we implemented a *h-index filter*. Which takes a specific a specific number to represent the h-index. Then applies a filter on the teams and then presents that graphically with the tournaments and teams that participated in them. This is based on the analytical tool by the same named used in research paper influence measurement. The first two queries are executed to gather the number of *Played In* edges each team has. If filtering by the number of wins the third query is executed to determine number of wins within the edges. If filtering by the participation the last query is executed to check the number of edges total. Figure 6 showcases the web pages them self, including both types of h-index filters.

```
SELECT SourceNodeID, COUNT(DISTINCT TargetNodeID)
AS participant_count
FROM Edges
WHERE EdgeType = "Played_In"
GROUP BY SourceNodeID
SELECT NodeID, Name, NodeType, Attributes
FROM Nodes
WHERE NodeType = "Team"
ORDER BY Name
```

```
SELECT e.EdgeID, e.SourceNodeID, e.TargetNodeID,
tr.Name AS tournament_name,
tr.Attributes AS tournament_attributes
FROM Edges e
JOIN Nodes tr ON tr.NodeID = e.SourceNodeID
WHERE e.EdgeType = "Won_By"
AND tr.NodeType = "Tournament"
ORDER BY tr.Name
```

```

SELECT e.EdgeID, e.SourceNodeID, e.TargetNodeID,
       tr.Name AS tournament_name,
       tr.Attributes AS tournament_attributes
FROM Edges e
JOIN Nodes tr ON tr.NodeID = e.SourceNodeID
WHERE e.EdgeType = "Played_In"
       AND tr.NodeType = "Tournament"
ORDER BY tr.Name

```

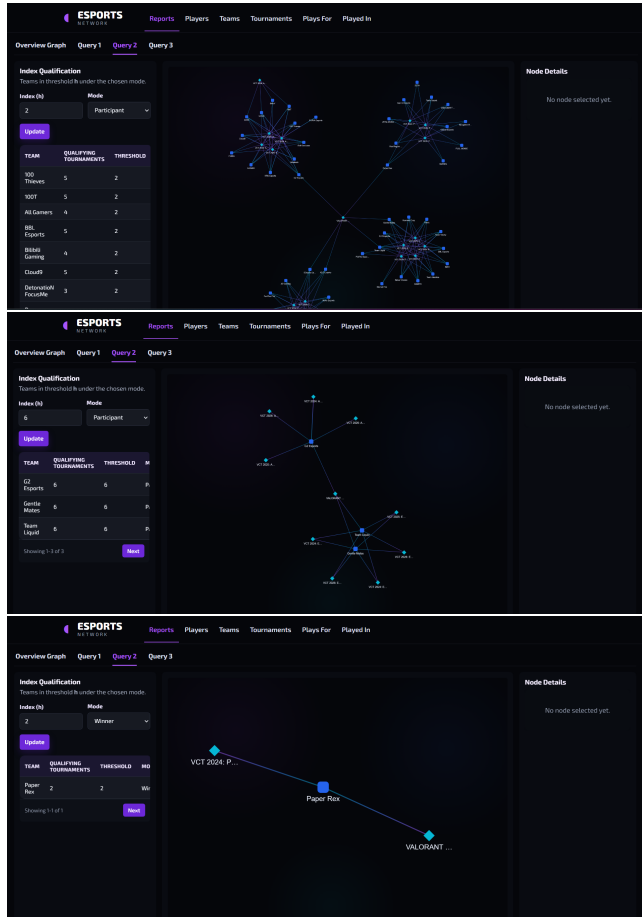


Figure 6: Query 2 Webpage with Examples

5.5 Query 3

The last analytical query we implemented is a *Q1 Tournament Influence Network*. Which is based on the *Q1 Journal Influence Network* used by research paper databases. We defined a Q1 tournament as a tournament where the prize pool is greater than 1 million dollars. The influence network of a Q1 tournament is all teams that participated in that tournament. For this query the below statements are executed. The first query gets all the tournaments from the database. The second gets the team nodes and the last query gets the respective edges. The tournaments are then filtered by the prize pool size. Figure 7 depicts the respective web page.

```

SELECT DISTINCT n.NodeID, n.Name, n.NodeType, n.Attributes
FROM Nodes n
JOIN GraphMemberships gm ON gm.NodeID = n.NodeID
WHERE gm.GraphID = 3
       AND n.NodeType = "Tournament"
ORDER BY n.Name

```

```

SELECT DISTINCT n.NodeID, n.Name, n.NodeType, n.Attributes
FROM Nodes n
JOIN GraphMemberships gm ON gm.NodeID = n.NodeID
WHERE gm.GraphID = 3
       AND n.NodeType = "Team"
ORDER BY n.Name

```

```

SELECT DISTINCT e.EdgeID, e.SourceNodeID, e.TargetNodeID,
       e.EdgeType
FROM Edges e
JOIN GraphMemberships gm ON gm.EdgeID = e.EdgeID
WHERE gm.GraphID = 3
       AND e.EdgeType IN ("Played_In", "Won_By")
ORDER BY e.EdgeID

```

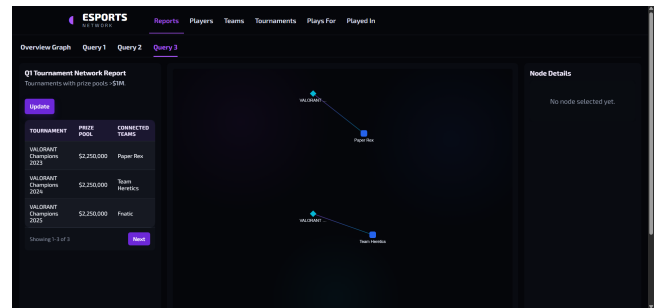


Figure 7: Query 3 Webpage with Example

5.6 Timeline Comparison

Below is the timeline outlined in phase one.

- Phase 1:
 - Initial Design Presentation
Date: March 3rd
 - Initial Project Outline Report
Date: March 7th
- Phase 2:
 - Working Prototype Presentation
Date: March 24th
 - General Design Report
Date: March 28th
- Phase 3:
 - Fully Function Prototype Presentation
Date: April 21st
 - Expanded Design Report
Date: April 25th
- Phase 4:
 - Final Project Demo
Date: May 7th

- Finalized Project Report
Date: May 10th

Since we have finished the project on time it's clear we more or less have matched this schedule. We started by outlining the project's central idea and the timeline itself which comprised phase one. Next we moved towards creating the initial prototype and database, allowing for the initial create, read, update, delete operations. This prototype also included initial Cytoscape integration on the webpage. This comprised phase 2. Once the initial prototype was completed we then overhauled the schema to what was discussed earlier in this report. This also included the first and second analytical query implementation. This phase included most of the learning curve as we familiarized with the unknown technology, Cytoscape. Similarly the first phases of the web scrappers were completed at this phase. That concluded phase three. Finally we made a final sprint to finish the last analytical query as well as create the rest of the web scrappers. However, we then completely overhauled the web scrapper as it was inefficient with how it accessed the url and html data. Finally, we polished the overall look of the project as well as smaller UI features. Which concluded phase four with the final product.

6 Conclusion

E-Sports has rapidly evolved into a global form of entertainment across titles such as *League of Legends*, *Counter-Strike*, and *Dota 2*. Despite this growth, the analytical infrastructure supporting

e-sports still lags behind traditional sports. The E-Sports Competitive Network directly addresses this limitation by introducing a relational database designed to model the relationships between players, teams, and tournaments across multiple games. These relationships are showcased through an updated relationship schema and tech stack. The E-Sports Competitive Network allows for mass population of it's database through a set of custom developed web scrappers and manual insertion through its web pages. The E-Sports Competitive Network features three key queries, the teammate network which showcases how players are related across teams. A h-index filter to showcase teams based on the number of tournaments they've participated in as well as won. Lastly, the Q1 journal influence network which showcases all teams that have participated in a tournament with a prize pool over 1 million dollars. The E-Sports Competitive Network lays the groundwork for more advanced analytical tools that can benefit coaches, analysts, and researchers.

References

- [1] Microsoft. 2020. Playwright for Python. <https://github.com/microsoft/playwright-python>.
 - [2] Paul Shannon, Andrew Markiel, Owen Ozier, Nitin S. Baliga, Jonathan T. Wang, Daniel Ramage, Nada Amin, Benno Schwikowski, and Trey Ideker. 2003. Cytoscape: A Software Environment for Integrated Models of Biomolecular Interaction Networks. *Genome Research* 13, 11 (2003), 2498–2504. <https://doi.org/10.1101/gr.1239303>
- [2] [1]